

Modern Application Development

Frameworks, Architecture, and Docker

-ANJAN ABEYKOON -

Presentation Agenda

01 Modern App Development

02 Monoliths vs Microservices

03 The Challenge of Scratch

04 What is a Framework?

05 Best Practices

06 Conclusion & Q&A

Monoliths to Microservices

The Monolith

- A monolithic application combines all functionality into one large codebase
- Changes in one module can impact the entire system unexpectedly

Microservices

- Microservices divide the system into smaller independent services
- Each service can be developed, tested, and deployed separately

The Challenge of Building Software from Scratch

01

Building applications from scratch requires writing every component manually

02

As systems grow, the amount of repetitive work increases significantly

03

More code naturally introduces more bugs and hidden issues

04

Lack of structure makes debugging and maintenance increasingly difficult



What is a Framework?

- A framework provides a pre-built structure for developing applications
- Developers focus on filling in functionality rather than building everything
- It enforces best practices and standard design patterns
- This reduces development time and improves consistency



Framework vs Library

The Library

- A library provides reusable functions that developers call when needed
- You are in control; you decide when to use the tools.

The Framework

- A framework controls the program flow and calls developer-written code
- Control shifts from the developer to the framework

This distinction changes how applications are designed and structured

Inversion of Control (IoC)

Traditional Programming

Developers control the sequence of execution

The Framework Way

Control shifts from the developer to the system
The framework determines when and how your code runs

“Don’t call us, we’ll call you” principle

Dependency Injection (DI)

Core Concept

- Many objects depend on other components to perform their tasks

Instead of creating dependencies, they receive them from external sources

Reduced Coupling

Reduces tight coupling between different parts of the system

System Benefits

Improves flexibility, testing, and maintainability

Convention over Configuration

The Framework Approach

- Frameworks provide default behaviors based on common naming conventions
- Developers follow standard patterns instead of writing detailed configurations
- This reduces the need for extensive setup code
- It allows faster development with fewer errors

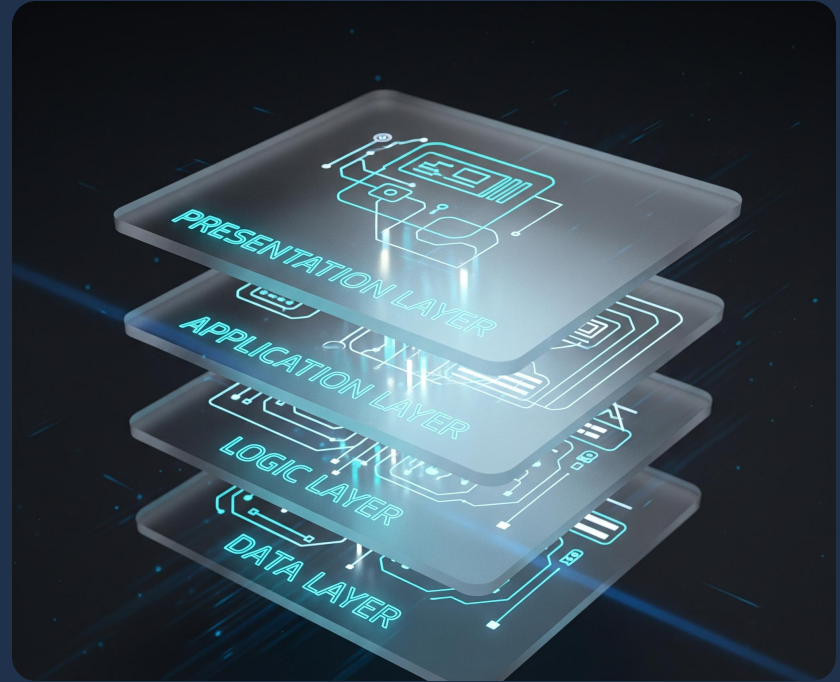


Optimizing efficiency through standardized architectural patterns.

Introduction to Layered Architecture

Core Principles

- Applications are divided into layers based on specific responsibilities
- Each layer is responsible for handling a particular type of task
- Layers interact in a structured and predictable manner
- Supports the principle of separation of concerns



Enhancing maintainability through structured separation of concerns.

The Model Layer

Data Structure & Entities

- Defines how data is structured within the application
- Represents entities and their complex relationships
- Independent of business logic and user interaction
- Ensures consistency in data handling across the entire system

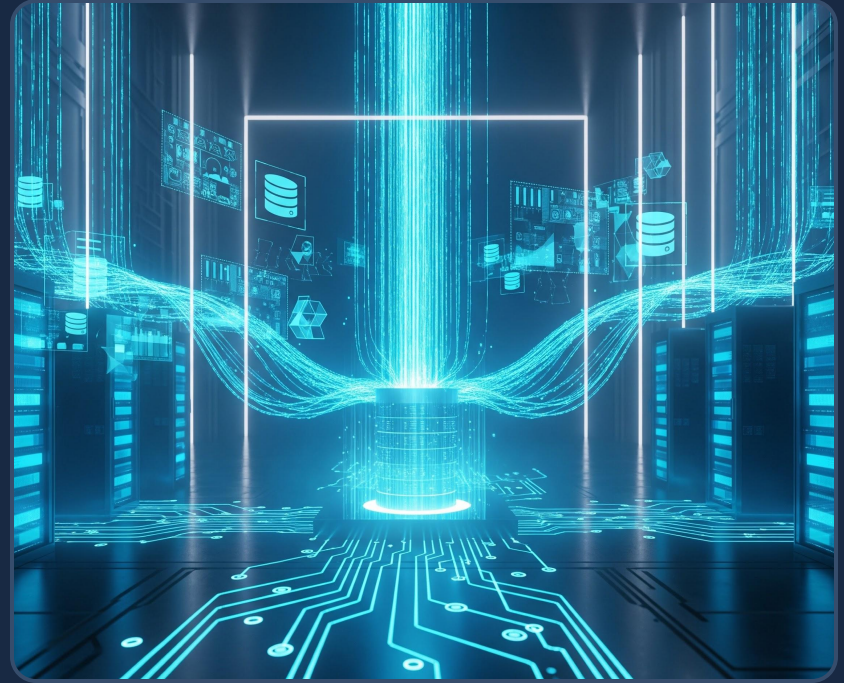


Establishing the foundation of data integrity within the layered architecture.

The Repository Layer

Data Persistence & Logic

- The repository layer manages communication with the database
- It handles storing, retrieving, and querying data
- This separates database logic from business logic
- Changes in the database have minimal impact on other layers

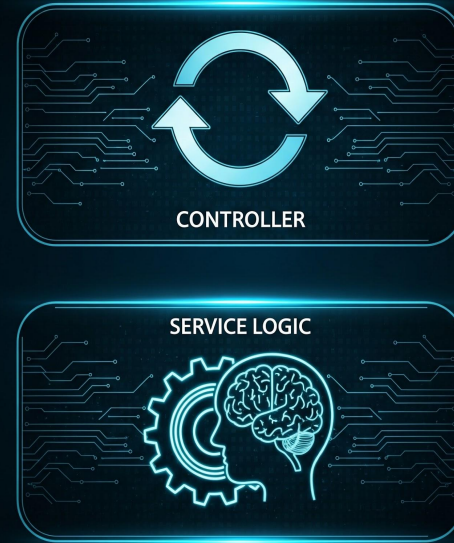


Decoupling data access to ensure architectural flexibility and clean code.

Service and Controller Layers

Business Logic & Routing

- The service layer contains the core business logic of the application
- The controller layer handles incoming requests and outgoing responses
- Controllers delegate processing tasks to the service layer
- This separation keeps logic and communication responsibilities distinct

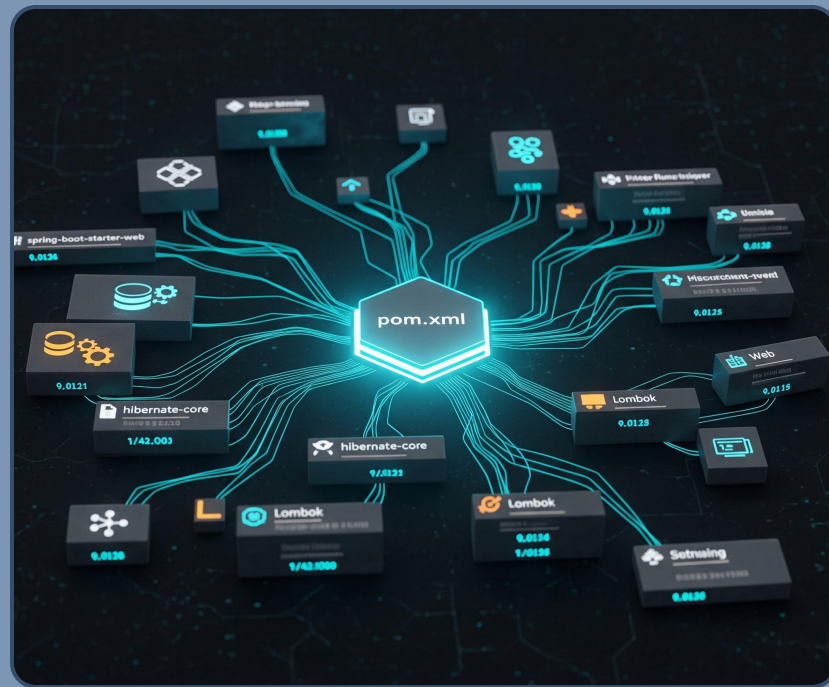


Streamlining request handling and business execution through modular design.

Maven – Managing Dependencies

Dependency Lifecycle

- Applications rely on external libraries to extend functionality
- Maven allows developers to define dependencies in a single configuration file
- It automatically downloads and manages required libraries
- Ensures consistent builds across different environments



Postman – API Testing

API Interaction & Validation

- Postman allows developers to test APIs without building a frontend
- It sends requests and displays responses from the server
- This helps verify backend functionality quickly
- It is especially useful during early development stages

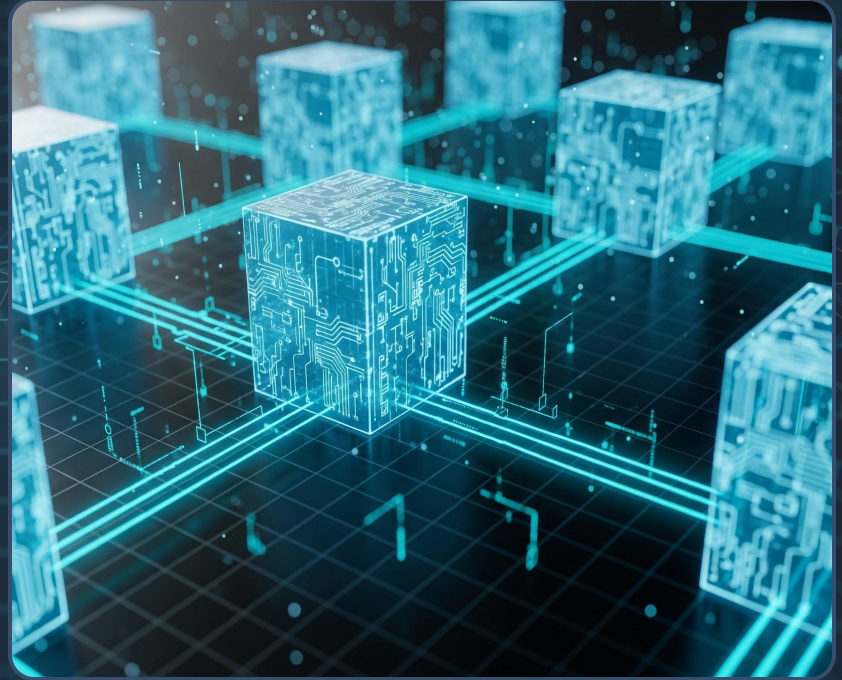


Accelerating backend validation through efficient and standalone API request simulation.

Docker – Containerization

Environment Isolation

- Docker enables applications to run inside lightweight containers
- A container includes the application and all its dependencies
- This ensures consistency across different systems
- It eliminates the “works on my machine” problem



Standardizing application deployment through immutable and portable container environments.

Summary – Bringing It All Together

Modern Software Ecosystem

- Frameworks provide structure and reduce development complexity
- IoC and DI improve flexibility and manage dependencies effectively
- Layered architecture separates responsibilities within applications
- Tools and Docker streamline development, testing, and deployment



Together, these practices define modern professional software development.